Modal Guitar Lab

Chapitre 03 — Sélection de la tonique et des gammes / modes

24 octobre 2026

Table des matières

1 Guide de lecture **2**

2 Résultat de la séance **2**

3 Situation de départ : acquis des séances 1 et 2 **2**

4 Décision centrale : distinguer MIDI et classe de hauteur **3**
4.1 Le problème rencontré . . . 3
4.2 Règle retenue . . . 3
4.3 Pourquoi utiliser `modulo()` ? . . . 3

5 Étape 1 — Représenter les douze notes **4**
5.1 Première proposition et limite découverte . . . 4
5.2 Liste immuable au niveau du typage . . . 4
5.3 Obtenir le nom depuis la classe de hauteur . . . 4

6 Étape 2 — Sélecteur de tonique contrôlé **4**
6.1 Le state et les options . . . 5
6.2 Difficulté réelle : un sélecteur visuellement désynchronisé . . . 5

7 Étape 3 — Relier la tonique aux couleurs du manche **5**
7.1 Pourquoi aucun `useEffect` ? . . . 5
7.2 Première synchronisation de `buildOutput()` . . . 6

8 Étape 4 — Décrire quatre gammes / modes **6**
8.1 Remplacer une constante unique par un dictionnaire . . . 6
8.2 Dériver le type des données . . . 6

9 Étape 5 — Généraliser le test d'appartenance **6**
9.1 Contrat de la nouvelle fonction . . . 6

10 Étape 6 — Gérer le mode dans React **7**
10.1 Un second state indépendant . . . 7
10.2 Vérifier le calcul avant d'ajouter le contrôle . . . 7
10.3 Ajouter le sélecteur de modes . . . 7

11 Étape 7 — Synchroniser le panneau de note **8**

12 Tests de recette reproductibles **9**
12.1 Vérifications fonctionnelles . . . 9
12.2 Validation de construction . . . 9
12.3 Tests automatisés proposés pour une séance ultérieure . . . 9

13 Journal des décisions et difficultés **9**
13.1 Ce qui relève encore d'un choix pragmatique . . . 10

14 Reconstruction depuis zéro — implémentation de référence **10**
14.1 Initialiser le projet 10
14.2 `src/music/scales.ts` — données et algorithmes 10
14.3 `src/music/notes.ts` — hauteur et nom 11
14.4 `src/GuitarNeck.tsx` — états, interactions et rendu 11
14.5 `src/GuitarNeck.css` — exemple minimal, non historique 14
14.6 `src/App.tsx` et vérification 15

15 Notions à retenir pour la suite **15**

16 Fin de chapitre et reprise de la séance 4 **16**

Statut : séance 3 terminée et validée fonctionnellement.
Rôle du document : troisième chapitre d'une documentation cumulative visant à expliquer et reconstruire Modal Guitar Lab depuis zéro.
Périmètre : sélection de la tonique et des quatre gammes/modes, sans audio, tablature, authentification ni backend.

1 Guide de lecture

Ce chapitre combine trois niveaux de documentation :

- **Historique vérifié de la séance** : choix réalisés, propositions de code soumises et tests confirmés pendant nos échanges.
- **Explication pédagogique** : concepts JavaScript, React, TypeScript et musicaux derrière ces choix.
- **Implémentation de référence reconstituée** (annexe) : ensemble de fichiers cohérent pour refaire fonctionner ce jalon sans avoir le dépôt d'origine sous les yeux. Le code de référence n'est **pas** une copie certifiée du dépôt : les fichiers complets `notes.ts`, les styles CSS et le branchement `App.tsx` n'avaient pas été fournis pendant la séance.

Les séances 1 et 2 ont été indiquées comme validées dans le contexte de départ, mais leurs sources complètes n'ont pas été revues ici. Le présent chapitre documente leurs **prérequis déclarés**, puis approfondit uniquement les décisions de la séance 3.

2 Résultat de la séance

Objectif initial. Remplacer une visualisation figée de Mi mineur naturel par une exploration interactive de **12 toniques** et **4 gammes ou modes**, avec mise à jour immédiate du manche.

Résultat observé et confirmé par l'utilisateur :

Comportement	Résultat
Tonique sélectionnable de C à B, demi-tons inclus	Validé
Modes mineur naturel, majeur, dorien, phrygien	Validé
Cases recolorées lorsque tonique ou mode changent	Validé
Toniques distinguées des autres notes de gamme	Validé
Position sélectionnée conservée lors d'un changement de mode	Validé dans le scénario testé
Indication « Dans la gamme / Hors gamme » recalculée	Validé
Compilation avec `pnpm build`	Confirmée lors de la séance

Ce qui n'a pas été ajouté : son, rythme, progression d'accords, tablature, état global, bibliothèque musicale, persistance et optimisation prématurée.

3 Situation de départ : acquis des séances 1 et 2

À l'ouverture de la séance 3, l'application possède déjà un manche de six cordes, des cases **0 à 12**, une position sélectionnée dans un `useState`, ainsi que des calculs de notes MIDI et de noms de

notes.

Accordage standard, de la première à la sixième corde :

Corde (`pos`)	Note à vide	Numéro MIDI
1	E4	64
2	B3	59
3	G3	55
4	D3	50
5	A2	45
6	E2	40

La case 0 représente la corde à vide. La hauteur d'une note à la case `fret` est `openMidi + fret`. La fonction `getMidiNote()` encapsule cette règle ; `getFullNoteName()` renvoie son nom avec octave.

La logique musicale est rangée dans `src/music/notes.ts` et `src/music/scales.ts`. Avant la séance, `scales.ts` contient une gamme mineure naturelle représentée par des intervalles, ainsi que `modulo()` et `isNoteInNaturalMinor()`. La tonique est encore **codée en dur à MIDI 64** dans le calcul des couleurs.

Code problématique au départ :

```
const tonicMidi = 64;
const isInScale = isNoteInNaturalMinor(midiNote, tonicMidi);
const isTonic = midiNote % 12 === tonicMidi % 12;
```

L'affichage associe vert aux notes appartenant à la gamme, bleu aux toniques et orange à la case sélectionnée. L'objectif n'est pas de refaire ce manche, mais de rendre sa logique paramétrable.

4 Décision centrale : distinguer MIDI et classe de hauteur

4.1 Le problème rencontré

Lors du passage de la tonique fixe `64` à un état numérique `4`, une confusion est apparue : **« Je ne vois pas comment retrouver le MIDI 64. »** Le blocage était pertinent, parce que deux concepts étaient implicitement mélangés jusque-là :

- un **numéro MIDI**, qui décrit une hauteur dans une octave précise ;
- une **classe de hauteur**, qui décrit la note indépendamment de l'octave.

La sélection d'une *gamme* demande de savoir si la note est un Mi, un Fa, etc., et non d'imposer Mi4 plutôt que Mi3.

4.2 Règle retenue

En tempérament égal à 12 demi-tons par octave, les classes de hauteur sont les entiers de **0 à 11** :

Classe	0	1	2	3	4	5	6	7	8	9	10
Note	C	C#	D	D#	E	F	F#	G	G#	A	A#

Par exemple, les notes MIDI **40, 52, 64 et 76** sont toutes des Mi car `midi % 12 === 4`.

On décide donc que le state React `tonic` contiendra une **classe de hauteur**, initialement `4`, et **non** un numéro MIDI absolu.

```
const [tonic, setTonic] = useState(4);
```

Ce choix simplifie les calculs musicaux et évite d'introduire artificiellement une octave de référence pour la tonique.

4.3 Pourquoi utiliser `modulo()` ?

Nous cherchons l'écart entre la note et la tonique, **ramené dans une octave** :

```
const distance = modulo(midiNote - tonic, 12);
```

Exemple : Sol3 a pour numéro MIDI `55`, Mi pour classe de hauteur `4` ; l'écart brut vaut `55 - 4 = 51`, mais `51 % 12 = 3`. La distance relative est donc **3 demi-tons**, soit la tierce mineure de Mi.

Autre exemple, vérifié durant la séance : La2 = `45` et tonique La = `9`. La distance vaut `modulo(45 - 9, 12) = 0` : c'est bien une tonique.

En JavaScript, le `%` est un opérateur de *reste*, pas un modulo mathématique toujours positif. La fonction suivante normalise le résultat dans l'intervalle `[0, base - 1]` :

```
export function modulo(value: number, base: number): number {
  return ((value % base) + base) % base;
}
```

Nous réutilisons cet outil plutôt que de dupliquer des formules dans le composant.

5 Étape 1 — Représenter les douze notes

5.1 Première proposition et limite découverte

Une première idée consistait à utiliser une union de sept notes naturelles et un `Partial<Record<NoteName, number>>` :

```
type NoteName = "C" | "D" | "E" | "F" | "G" | "A" | "B";
type Tonic = Partial<Record<NoteName, number>>;
// Ex. { E: 4 }
```

Cette structure est légitime pour une **table de correspondance**, mais pas comme modèle d'**une sélection unique** : elle autorise `{}`, `{ C: 0, E: 4 }`, etc. De plus, elle oublie les altérations chromatiques.

Décision : séparer la liste de référence des notes du seul nombre à stocker dans le state.

5.2 Liste immuable au niveau du typage

```
export const CHROMATIC_SCALE = [
  "C", "C#", "D", "D#", "E", "F",
  "F#", "G", "G#", "A", "A#", "B"
] as const;

export type NoteName = (typeof CHROMATIC_SCALE)[number];
```

Pourquoi `as const` ? Un tableau ordinaire est inféré comme `string[]`. Avec `as const`, TypeScript connaît les douze chaînes littérales et les positions du tuple, en lecture seule **au niveau du système de types**. Ce n'est pas un `Object.freeze()` exécuté par JavaScript.

`typeof CHROMATIC_SCALE` récupère le type du tuple ; `[number]` en extrait l'union des valeurs possibles. On évite ainsi d'écrire et maintenir une liste de noms de notes à deux endroits.

5.3 Obtenir le nom depuis la classe de hauteur

```
export function getTonicName(tonic: number): NoteName {
  const tonicName = CHROMATIC_SCALE[modulo(tonic, 12)];
  return tonicName ?? "C";
}
```

Le cas `getTonicName(16)` renvoie `E` puisque `16 modulo 12 = 4`.

Point de vigilance. `?? "C"` fournit une valeur de repli lorsque l'accès retourne `undefined` ; il **ne valide pas** qu'une entrée est cohérente. Dans cette séance, le state est alimenté exclusivement par une sélection contrôlée. Une validation plus explicite deviendrait préférable si la tonique provenait ultérieurement d'un fichier, d'une URL ou d'une API.

6 Étape 2 — Sélecteur de tonique contrôlé

6.1 Le state et les options

Le composant `GuitarNeck` conserve la tonique :

```
const [tonic, setTonic] = useState(4);
```

Une boucle `CHROMATIC_SCALE.map(...)` génère les options sans recopier les douze noms. Dans le code réalisé durant la séance, chaque `<option>` utilise le **nom** comme `value` :

```
<select
  id="tonic-select"
  name="tonic"
  value={getTonicName(tonic)}
  onChange={handleTonicChange}
>
  {CHROMATIC_SCALE.map((name) => (
    <option key={name} value={name}>{name}</option>
  ))}
</select>
```

Le gestionnaire convertit le nom sélectionné en classe de hauteur grâce à sa position dans le tuple :

```
const handleTonicChange = (
  event: ChangeEvent<HTMLSelectElement>
) => {
  const name = event.target.value as NoteName;
  const pitchClass = CHROMATIC_SCALE.indexOf(name);
  if (pitchClass >= 0) setTonic(pitchClass);
};
```

`event.target.value` est une `string`, même quand les valeurs ressemblent à des nombres. L'assertion `as NoteName` informe le compilateur, mais **n'est pas une validation au runtime**. Le garde `pitchClass >= 0` dans ce code de référence est une petite précaution ajoutée pour la reconstruction ; le gestionnaire initial montrait directement `setTonic(tonicNumber)`.

6.2 Difficulté réelle : un sélecteur visuellement désynchronisé

La première version du `<select>` possédait `onChange`, mais **pas** de propriété `value`.

Au chargement : le state valait `4` (Mi) tandis que le navigateur montrait par défaut **C**, première option de la liste. Le texte affichant la tonique disait **E**. Ajouter `value={getTonicName(tonic)}` a supprimé cette divergence.

Principe React : dans un composant contrôlé, le state est la source de vérité. La valeur du sélecteur provient du state, et `onChange` demande la mise à jour de celui-ci.

7 Étape 3 — Relier la tonique aux couleurs du manche

Le premier changement à apporter dans la boucle des cases était volontairement minimal :

```
const isInScale = isNoteInNaturalMinor(midiNote, tonic);
const isTonic = midiNote % 12 === tonic;
```

On élimine la constante `tonicMidi = 64`. Le paramètre historique de `isNoteInNaturalMinor()` pouvait recevoir directement une classe de hauteur puisque le calcul d'intervalle utilisait un modulo 12.

Tests manuels confirmés : La mineur naturel contient A, B, C, D, E, F, G ; Do mineur naturel contient C, D, D# (= Eb), F, G, G# (= Ab), A# (= Bb). Le changement de couleur fonctionnait sans rechargement.

7.1 Pourquoi aucun `useEffect` ?

`setTonic()` demande à React un nouveau rendu de `GuitarNeck()`. Pendant ce rendu, les `.map()` qui construisent les cases s'exécutent avec la nouvelle valeur de `tonic` ; les classes CSS sont recalculées.

La présence d'un calcul dérivé de state **ne justifie pas** un `useEffect`, ni un deuxième state du

type `coloredFrets`. Ajouter cet état dérivé créerait un risque de désynchronisation et du code de coordination inutile.

7.2 Première synchronisation de `buildOutput()`

L'aide « Dans la gamme / Hors gamme » contenait encore `isNoteInNaturalMinor(midi, 64)`. Il a fallu lui transmettre la tonique comme **argument**, parce que `buildOutput()` est une fonction définie **hors** de `GuitarNeck` et ne peut pas accéder à ses variables locales.

Transmettre explicitement les données rend la fonction indépendante de React et facilite la compréhension de ses entrées. Ce n'est pas une nécessité absolue de la placer hors composant, mais ce découpage est simple et cohérent à ce stade.

8 Étape 4 — Décrire quatre gammes / modes

8.1 Remplacer une constante unique par un dictionnaire

La gamme mineure naturelle était initialement :

```
const NATURAL_MINOR_INTERVALS = [0, 2, 3, 5, 7, 8, 10];
```

Nous souhaitons désormais retrouver une gamme à partir de son identifiant : `SCALES["dorian"]`.

```
export const SCALES = {
  dorian:   [0, 2, 3, 5, 7, 9, 10],
  phrygian: [0, 1, 3, 5, 7, 8, 10],
  major:    [0, 2, 4, 5, 7, 9, 11],
  minor:    [0, 2, 3, 5, 7, 8, 10]
} as const;
```

Une petite erreur de frappe, `prygian`, a été corrigée en `phrygian`. La vigilance sur les identifiants est importante : ils constituent des valeurs utilisées à la fois par les données, les types et l'interface.

Mode	Formule en demi-tons depuis la tonique	Couleur caractéristique
Mineur naturel	0, 2, 3, 5, 7, 8, 10	Sixte mineure
Majeur	0, 2, 4, 5, 7, 9, 11	Tierce majeure
Dorien	0, 2, 3, 5, 7, 9, 10	Sixte majeure sur une base mineure
Phrygien	0, 1, 3, 5, 7, 8, 10	Seconde mineure

Ces tableaux contiennent **des distances en demi-tons**, pas des notes MIDI et pas les indices d'un tableau de noms de notes.

8.2 Dériver le type des données

```
export type ScaleName = keyof typeof SCALES;
```

Le type obtenu est l'union `"dorian" | "phrygian" | "major" | "minor"`. L'avantage, identifié pendant la séance : **si `SCALES` évolue, `ScaleName` évolue automatiquement**. Une seule liste fait foi.

9 Étape 5 — Généraliser le test d'appartenance

9.1 Contrat de la nouvelle fonction

```
isNoteInScale(midiNote, tonic, scaleName): boolean
```

On calcule la distance de la note à la tonique, puis on recherche cette distance parmi les intervalles du mode.

Première implémentation proposée :

```
export function isNoteInScale(
  midiNote: number,
  tonic: number,
```

```
  scaleName: ScaleName
): boolean {
  const distance = modulo(midiNote - tonic, 12);
  return SCALES[scaleName].find(
    (el) => el === distance
  ) !== undefined ? true : false;
}
```

Elle fonctionne. Le test explicite `!== undefined` est essentiel si on utilise `find()`, puisque **0** est un intervalle valide mais *falsy*.

Version retenue, plus expressive :

```
export function isNoteInScale(
  midiNote: number,
  tonic: number,
  scaleName: ScaleName
): boolean {
  const distance = modulo(midiNote - tonic, 12);
  return SCALES[scaleName].some(
    (interval) => interval === distance
  );
}
```

`find()` cherche un **élément** (`number | undefined`) ; `some()` répond à une **question existentielle** (`boolean`). Les deux s'arrêtent dès qu'une correspondance est trouvée. L'intérêt est la lisibilité plutôt qu'un gain sensible de performance.

9.1.1 Subtilité TypeScript liée à `as const`

`SCALES[scaleName]` est un tuple readonly dont les valeurs ont des types littéraux. Suivant l'inférence et les unions de tuples, `.includes(distance)` peut occasionner une erreur quand `distance` est un simple `number`. Le callback de `.some()` compare sans exiger que son argument soit lui-même une valeur littérale autorisée par la signature d'`includes`.

9.1.2 Une confusion pédagogique utile

L'expression `SCALES["minor"][3]` retourne le **quatrième élément** du tableau, soit `5`. Elle ne signifie pas « l'intervalle 3 existe ». Pour tester la présence de l'intervalle `3`, utiliser `.some(interval => interval === 3)` ou une recherche équivalente.

10 Étape 6 — Gérer le mode dans React

10.1 Un second state indépendant

```
const [scaleName, setScaleName] = useState<ScaleName>("minor");
```

Le state du mode reste distinct de la tonique. Ils changent indépendamment, mais ensemble déterminent les mêmes calculs de coloration :

```
const isInScale = isNoteInScale(midiNote, tonic, scaleName);
```

Cette séparation est suffisante : pas de `useReducer`, de Context ou de bibliothèque d'état globale pour deux variables locales.

10.2 Vérifier le calcul avant d'ajouter le contrôle

Le premier test a consisté à changer temporairement l'état initial de `"minor"` à `"phrygian"`, puis à `"dorian"`. En **Mi phrygien**, Fa naturel entre dans la gamme et Fa# en sort. On prouve d'abord la validité de l'algorithme, avant de chercher d'éventuels problèmes de JSX.

10.3 Ajouter le sélecteur de modes

Les options sont dérivées des propriétés de `SCALES` :

```
<select
  name="mode"
  id="mode-select"
  value={scaleName}
  onChange={handleScaleChange}
>
  {Object.keys(SCALES).map((scale) => (
    <option key={scale} value={scale}>{scale}</option>
  ))}
</select>
```

`Object.keys()` retourne **`string[]`**, pas `ScaleName[]`. Cela ne pose aucun problème pour produire les attributs `key` et `value` des éléments JSX.

Le gestionnaire soumis pendant la séance était :

```
const handleScaleChange = (
  event: ChangeEvent<HTMLSelectElement>
) => {
  const selectedScale = event.target.value as ScaleName;
  setScaleName(selectedScale);
};
```

L'assertion ne contrôle pas les données à l'exécution. Elle est acceptable pour ce premier sélecteur, alimenté à partir de l'objet `SCALES`. Si le mode pouvait être reçu de l'extérieur, il faudrait vérifier qu'il s'agit bien d'une clé existante.

Un détail d'interface a également été corrigé : le `label` de ce **deuxième** sélecteur indiquait encore « Choisissez la Tonique ». Il doit mentionner « gamme / mode ».

11 Étape 7 — Synchroniser le panneau de note

Le statut de la note sélectionnée devait dépendre **des deux états**. La fonction finale soumise était :

```
function buildOutput(
  selectedPosition: SelectedPosition,
  selected: GuitarString,
  tonic: number,
  scaleName: ScaleName
) {
  const midi = getMidiNote(
    selected.openMidi,
    selectedPosition.fret
  );
  const note = getFullNoteName(midi);
  const scaleStatus = isNoteInScale(midi, tonic, scaleName)
    ? "Dans la gamme"
    : "Hors gamme";

  return <span>{
    `NOTE: ${note} MIDI: ${midi} ${scaleStatus} ` +
    `SCALE: ${getTonicName(tonic)} ${scaleName}`
  }</span>;
}
```

La fonction reçoit maintenant **position, corde, tonique, mode** ; son appel JSX est devenu :

```
{selectedPosition && selected
  ? buildOutput(selectedPosition, selected, tonic, scaleName)
  : <span>Aucune note sélectionnée</span>}
```

Conséquence importante : **changer de mode sans recliquer sur la case** met à jour le message. L'identité de la position sélectionnée est indépendante de sa lecture dans un contexte harmonique donné.

12 Tests de recette reproductibles

12.1 Vérifications fonctionnelles

Contexte	Note testée	Résultat attendu
E mineur naturel	F4, MIDI 65	Hors gamme
E phrygien	F4, MIDI 65	Dans la gamme
E dorien	F4, MIDI 65	Hors gamme
E majeur	F4, MIDI 65	Hors gamme
E mineur naturel	F#4, MIDI 66	Dans la gamme
E majeur	G#4, MIDI 68	Dans la gamme
E dorien	C# (n'importe quelle octave)	Dans la gamme
A mineur naturel	A2, MIDI 45	Tonique
C mineur naturel	D#, G#, A#	Dans la gamme

Protocole conseillé : démarrer avec E mineur, sélectionner la case **1 de la corde 1** (F4/MIDI 65), basculer entre les quatre modes, observer à la fois la case et le texte « Dans/Hors gamme », **sans cliquer à nouveau** sur F4. Tester ensuite A et C comme toniques. La priorité visuelle doit rester : sélection orange, puis tonique bleue, puis note de gamme verte.

12.2 Validation de construction

```
pnpm build
```

Le succès du build a été confirmé durant les exercices ; la vérification finale du comportement a également reçu confirmation. Les échanges ne contiennent pas de capture du log complet d'un test automatisé. **Ne pas confondre** compilation réussie et couverture de tests unitaires : aucun jeu de tests automatisés n'a été produit pendant la séance.

12.3 Tests automatisés proposés pour une séance ultérieure

Le calcul musical est pur et se prête à quelques tests ciblés :

```
import { strict as assert } from "node:assert";
import { isNoteInScale } from "./src/music/scales";

assert.equal(isNoteInScale(65, 4, "minor"), false);
assert.equal(isNoteInScale(65, 4, "phrygian"), true);
assert.equal(isNoteInScale(66, 4, "dorian"), true);
assert.equal(isNoteInScale(45, 9, "minor"), true);
```

Ce bloc exprime **des assertions de référence**, non une suite de tests déjà installée ou exécutable telle quelle via le runner du projet. Un futur chapitre pourra les intégrer au système de tests choisi.

13 Journal des décisions et difficultés

Situation	Décision	Raison / enseignement
`Partial<Record<NoteName, number>>` pour une tonique	Remplacer par `number` + liste chromatique	Une tonique unique n'est pas une table à clés facultatives
Sept notes naturelles seulement	Étendre à douze notes	Couvrir toute la gamme chromatique
Liste + type doublonnés	`as const` et `(typeof ...)[number]`	Dériver les types des données
Tonique MIDI `64` difficile à « retrouver »	Stocker la classe `4`	Éviter l'octave superflue
`select` initialement sans `value`	Ajouter la propriété contrôlée	Synchroniser DOM et React
Couleurs codées en dur pour Mi mineur	Utiliser `tonic` puis `scaleName`	Rendu calculé depuis l'état courant
Statut de note encore lié à `64`	Passer les paramètres à `buildOutput()`	Garder l'affichage cohérent
Quatre gammes potentiellement dupliquées	Objet `SCALES`, type dérivé `keyof typeof`	Extensibilité sans double maintenance

Situation	Décision	Raison / enseignement
`find() !== undefined ? true : false`	Préférer `some()`	Exprimer clairement un test booléen
`Object.keys` et valeur des `<select>` typés `string`	Assertion locale, options générées en interne	Comprendre limite entre TS statique et runtime
Besoin de recolorer le manche	Aucun `useEffect`	Le rerender suffit pour les valeurs dérivées

13.1 Ce qui relève encore d'un choix pragmatique

- **Notation enharmonique.** On affiche des dièses (`D#`, `G#`, `A#`), même lorsque la théorie écrirait plutôt `Eb`, `Ab`, `Bb` : choix de simplicité, à reconsidérer selon le contexte harmonique.
- **Type de la tonique.** `number` ne garantit pas à lui seul la plage `[0, 11]` ; le sélecteur fournit aujourd'hui les seules valeurs possibles.
- **Validation des modes.** Une assertion `as ScaleName` n'est pas une vérification runtime.
- **Position de `buildOutput`.** Fonction extérieure sans architecture de composants supplémentaire. Un futur découpage n'est utile que s'il répond à un besoin réel.

14 Reconstruction depuis zéro — implémentation de référence

Important. Les fichiers qui suivent constituent une **version pédagogique reconstituée** à partir des signatures, extraits et comportements confirmés. Ils peuvent servir de point de départ autonome ; ils ne prétendent pas représenter au caractère près les fichiers locaux de la séance.

14.1 Initialiser le projet

Dans un terminal disposant de Node.js et de pnpm :

```
pnpm create vite modal-guitar-lab --template react-ts
cd modal-guitar-lab
pnpm install
pnpm dev
```

Conserver le `tsconfig` strict généré par Vite et les fichiers d'amorçage habituels. Créer l'arborescence :

```
src/
  App.tsx
  GuitarNeck.tsx
  GuitarNeck.css
  music/
    notes.ts
    scales.ts
```

14.2 `src/music/scales.ts` — données et algorithmes

```ts
export const CHROMATIC_SCALE = [
  "C", "C#", "D", "D#", "E", "F",
  "F#", "G", "G#", "A", "A#", "B"
] as const;

export type NoteName = (typeof CHROMATIC_SCALE)[number];

export const SCALES = {
  dorian:   [0, 2, 3, 5, 7, 9, 10],
  phrygian: [0, 1, 3, 5, 7, 8, 10],
  major:    [0, 2, 4, 5, 7, 9, 11],
  minor:    [0, 2, 3, 5, 7, 8, 10]
} as const;
```

```ts
export type ScaleName = keyof typeof SCALES;

export function modulo(value: number, base: number): number {
  return ((value % base) + base) % base;
}

export function getTonicName(tonic: number): NoteName {
  const note = CHROMATIC_SCALE[modulo(tonic, 12)];
  return note ?? "C";
}

export function isNoteInScale(
  midiNote: number,
  tonic: number,
  scaleName: ScaleName
): boolean {
  const distance = modulo(midiNote - tonic, 12);
  return SCALES[scaleName].some(
    (interval) => interval === distance
  );
}

// Compatibilité pédagogique avec les séances précédentes.
export const NATURAL_MINOR_INTERVALS = SCALES.minor;

export function isNoteInNaturalMinor(
  midiNote: number,
  tonic: number
): boolean {
  return isNoteInScale(midiNote, tonic, "minor");
}
```

La fonction de compatibilité en bas est une **proposition pour faciliter la reconstruction incrémentale** ; le code achevé de la séance emploie `isNoteInScale()`.

14.3 `src/music/notes.ts` — hauteur et nom

```ts
import { CHROMATIC_SCALE, modulo } from "./scales";

export function getMidiNote(
  openMidi: number,
  fret: number
): number {
  return openMidi + fret;
}

export function getFullNoteName(midi: number): string {
  const note = CHROMATIC_SCALE[modulo(midi, 12)];
  const octave = Math.floor(midi / 12) - 1;
  return `${note}${octave}`;
}
```

Ce fichier est **reconstitué**, car ses corps de fonctions n’ont pas été fournis. Sa convention d’octave est cohérente avec la séance : MIDI 60 = C4 et MIDI 64 = E4.

14.4 `src/GuitarNeck.tsx` — états, interactions et rendu

```tsx
import { useState } from "react";
import type { ChangeEvent, CSSProperties } from "react";
import { getFullNoteName, getMidiNote } from "./music/notes";
import {
  CHROMATIC_SCALE,
  getTonicName,
```

```
  isNoteInScale,
  SCALES,
  type NoteName,
  type ScaleName
} from "./music/scales";
import "./GuitarNeck.css";

type GuitarString = {
  pos: 1 | 2 | 3 | 4 | 5 | 6;
  openMidi: number;
};

type SelectedPosition = {
  stringPos: GuitarString["pos"];
  fret: number;
};

function buildOutput(
  selectedPosition: SelectedPosition,
  selected: GuitarString,
  tonic: number,
  scaleName: ScaleName
) {
  const midi = getMidiNote(
    selected.openMidi,
    selectedPosition.fret
  );
  const note = getFullNoteName(midi);
  const scaleStatus = isNoteInScale(midi, tonic, scaleName)
    ? "Dans la gamme"
    : "Hors gamme";

  return (
    <p className="note-output">
      {`NOTE: ${note} MIDI: ${midi} ${scaleStatus} `}
      {`SCALE: ${getTonicName(tonic)} ${scaleName}`}
    </p>
  );
}

export default function GuitarNeck() {
  const [selectedPosition, setSelectedPosition] =
    useState<SelectedPosition | null>(null);
  const [tonic, setTonic] = useState(4);
  const [scaleName, setScaleName] =
    useState<ScaleName>("minor");

  const guitarStrings: GuitarString[] = [
    { pos: 1, openMidi: 64 },
    { pos: 2, openMidi: 59 },
    { pos: 3, openMidi: 55 },
    { pos: 4, openMidi: 50 },
    { pos: 5, openMidi: 45 },
    { pos: 6, openMidi: 40 }
  ];

  const fretNumbers = Array.from(
    { length: 13 }, (_, index) => index
  );

  const selected = guitarStrings.find(
```

```
  (string) => string.pos === selectedPosition?.stringPos
);

const handleTonicChange = (
  event: ChangeEvent<HTMLSelectElement>
) => {
  const name = event.target.value as NoteName;
  const pitchClass = CHROMATIC_SCALE.indexOf(name);
  if (pitchClass >= 0) setTonic(pitchClass);
};

const handleScaleChange = (
  event: ChangeEvent<HTMLSelectElement>
) => {
  const selectedScale = event.target.value as ScaleName;
  setScaleName(selectedScale);
};

return (
  <section className="guitar-lab">
    <div className="options-container">
      <label htmlFor="tonic-select">Tonique</label>
      <select
        id="tonic-select"
        value={getTonicName(tonic)}
        onChange={handleTonicChange}
      >
        {CHROMATIC_SCALE.map((name) => (
          <option key={name} value={name}>{name}</option>
        ))}
      </select>

      <label htmlFor="mode-select">Gamme / Mode</label>
      <select
        id="mode-select"
        value={scaleName}
        onChange={handleScaleChange}
      >
        {Object.keys(SCALES).map((scale) => (
          <option key={scale} value={scale}>
            {scale}
          </option>
        ))}
      </select>
    </div>

    <div className="g-neck-container">
      <div className="fretboard-headers">
        <div className="header-cell" />
        {fretNumbers.map((fret) => (
          <div key={fret} className="header-cell">
            {fret}
          </div>
        ))}
      </div>

      <div className="fretboard-grid">
        {guitarStrings.map((string) => (
          <div key={string.pos} className="string-row">
            <div className="string-name">
              {getFullNoteName(string.openMidi)}
```

```
            </div>
            {fretNumbers.map((fret) => {
              const midiNote = getMidiNote(
                string.openMidi, fret
              );
              const isSelected = selectedPosition !== null &&
                selectedPosition.stringPos === string.pos &&
                selectedPosition.fret === fret;
              const isInScale = isNoteInScale(
                midiNote, tonic, scaleName
              );
              const isTonic = midiNote % 12 === tonic;
              const note = isSelected
                ? getFullNoteName(midiNote)
                : "";

              return (
                <div
                  key={fret}
                  className={[
                    "fret-cell",
                    isInScale ? "in-scale" : "",
                    isTonic ? "is-tonic" : "",
                    isSelected ? "is-selected" : ""
                  ].filter(Boolean).join(" ")}
                  data-note={note}
                  onClick={() => setSelectedPosition({
                    stringPos: string.pos,
                    fret
                  })}
                >
                  <div
                    className="string-line"
                    style={{ "--i": string.pos } as CSSProperties}
                  />
                </div>
              );
            })}
          </div>
        ))}
      </div>
    </div>

    {selectedPosition && selected
      ? buildOutput(
          selectedPosition, selected, tonic, scaleName
        )
      : <p>Aucune note sélectionnée</p>}
  </section>
);
}
```

La comparaison d'une case sélectionnée est écrite ici directement dans le `.map()` plutôt que dans la fonction `cellIsSelected()` du code initial. Les deux approches produisent le même résultat ; ce changement sert uniquement à rendre l'exemple autonome et compact.

14.5 `src/GuitarNeck.css` — exemple minimal, non historique

La séance ne contenait pas la feuille de style originale. Celle-ci est donc **illustrative** et vise à rendre visibles les couleurs et le manche sans introduire de framework CSS.

```
.guitar-lab { font-family: system-ui, sans-serif; }
```

```css
.options-container {
  display: flex; align-items: center;
  gap: 0.75rem; flex-wrap: wrap;
  margin-bottom: 1rem;
}
.options-container select { padding: 0.4rem; }
.g-neck-container { overflow-x: auto; }
.fretboard-headers, .string-row {
  display: grid;
  grid-template-columns: 62px repeat(13, minmax(34px, 1fr));
  min-width: 570px;
}
.header-cell, .string-name {
  padding: 0.5rem 0.15rem;
  text-align: center;
}
.fret-cell {
  position: relative; min-height: 44px;
  border: 1px solid #9ca3af;
  background: #f1f5f9; cursor: pointer;
}
.fret-cell.in-scale { background: #97d4ad; }
.fret-cell.is-tonic { background: #88b9ec; }
.fret-cell.is-selected { background: #f3a85a; }
.string-line {
  position: absolute; top: 50%; left: 0; right: 0;
  height: 2px; /* Épaisseur simplifiée pour cet exemple */
  background: #475569;
}
.fret-cell.is-selected::after {
  content: attr(data-note);
  position: absolute; inset: 0;
  display: grid; place-items: center;
  font-size: 0.75rem; font-weight: 700;
}
.note-output { margin-top: 1rem; }
```

Les **règles CSS sont ordonnées** afin que la classe `is-selected` l'emporte sur `is-tonic`, elle-même prioritaire sur `in-scale`. L'ordre des classes dans l'attribut HTML ne garantit pas cette priorité : c'est la cascade CSS qui la détermine.

14.6 `src/App.tsx` et vérification

```tsx
import GuitarNeck from "./GuitarNeck";

export default function App() {
  return <GuitarNeck />;
}
```

Si les styles Vite par défaut interfèrent avec l'affichage, les retirer ou les ajuster dans la feuille globale. Puis :

```
pnpm dev
pnpm build
```

Vérifier le navigateur avec le protocole de recette de ce chapitre. Ne pas considérer ce code reconstitué comme testé tant qu'il n'a pas été effectivement compilé dans votre environnement local.

15 Notions à retenir pour la suite

Source de vérité. `SCALES` définit les intervalles et `ScaleName` est dérivé de ses clés ; `CHROMATIC_SCALE` définit les notes et `NoteName` est dérivé de ses éléments. On réduit les duplications et les incohé-

rences.

État minimal. `tonic`, `scaleName` et `selectedPosition` représentent les choix réels de l'utilisateur. L'appartenance d'une case à une gamme est **calculée** ; elle ne nécessite pas de state séparé.

Fonctions pures. Les calculs MIDI, modulo et appartenance aux gammes sont indépendants de React, ce qui les rend réutilisables et testables.

Frontend accessible à la compréhension. Deux `<select>` natifs, des `.map()` et quelques classes CSS suffisent ici ; on diffère les abstractions tant qu'elles n'apportent pas d'avantage concret.

Limites explicites. Le typage strict ne remplace pas la validation de données externes ; la notation dièse seule ne résout pas toutes les écritures harmoniques ; la coloration ne constitue pas encore un modèle de solo ou une tablature.

16 Fin de chapitre et reprise de la séance 4

Acquis stabilisés : exploration de 12 toniques et de 4 modes, calcul musical modulo 12, deux contrôles React, coloration dérivée des états et description de la note sélectionnée cohérente avec le contexte musical.

Sujet proposé pour le prochain chapitre : afficher les **intervalles/degrés musicaux** (1, 2, ♭3, 4, 5, 6, ♭7…) sur le manche, en distinguant clairement une distance chromatique en demi-tons et le nom d'un intervalle musical. Préserver l'architecture légère et la possibilité de choisir entre noms de notes et intervalles.

Prompt de reprise à copier dans un nouveau chat :

> Modal Guitar Lab — Séance 4 : visualisation des intervalles. Les trois premières séances sont validées. Nous avons un manche de 6 cordes (0–12), les calculs MIDI, une sélection de position, 12 toniques, quatre modes (`minor`, `major`, `dorian`, `phrygian`), les fonctions `modulo`, `getTonicName`, `isNoteInScale` et les états `tonic`, `scaleName`, `selectedPosition`. Le manche et `buildOutput()` sont synchronisés. Je veux progresser en petits exercices en écrivant moi-même le code. Commence par me faire réfléchir à la représentation TypeScript des intervalles musicaux, avant de les afficher sur le manche. À la fin, produire le chapitre 04 de la documentation progressive, avec décisions, étapes, difficultés, code de référence et tests de reconstruction.

Modal Guitar Lab — documentation progressive, chapitre 03. Établi à partir des échanges et du document de démarrage du projet, puis complété par une proposition d'implémentation de référence explicitement identifiée.